karaokame

Formato del cfg, formato del txt de UltraStar y configuración del script

karaokame genera canciones para UltraStar WorldParty a partir de la canción completa y su letra. Usa modelos locales: MDX23C (Ultimate Vocal Remover) para separar la voz del acompañamiento, Whisper para saber cuándo se canta cada palabra, CREPE para el tono de cada nota y Qwen (servido por Ollama) para proponer los metadatos y revisar las líneas dudosas. Este documento describe lo que el script lee, lo que escribe y cómo se configura.

1. Fichero de configuración de la canción (.cfg)

Cada canción se describe con un fichero de texto en `config/`. Tiene una línea por campo con la forma `clave="valor"`. Las comillas son opcionales, las líneas vacías y las que empiezan por # se ignoran, y las claves no distinguen mayúsculas.

```
name="Fenesta_che_lucive.txt"
audio="/home/roldan/karaokame/data/Fenesta_che_lucive.mp3"
lead=
accompaniment=
lyrics="/home/roldan/karaokame/data/Fenesta_che_lucive.txt"
language=
intrument=
score=
range="do3_mi4"
```

Lanzado sin argumentos, el script crea este fichero él mismo a partir de la canción que haya en `data/` (ver la sección 3). Solo hacen falta el nombre y la canción completa. El script genera el resto y lo escribe en el propio cfg: `lead`, `accompaniment`, `language` y, si el audio los contiene, `cover` o `video`. Las demás líneas no se tocan.

Campos principales

Campo	Contenido	Notas
`name`	Nombre base de los ficheros de salida.	Obligatorio. Si acaba en `.txt` se quita. Con el ejemplo se generan `songs/Fenesta_che_lucive.txt` y `.mp3`.
`audio`	Canción completa, con voz y música.	Cualquier formato con audio: mp3, wav, flac, m4a o un vídeo mp4, mkv, webm. Necesario si faltan `lead` y `accompaniment`: de él se separan las dos pistas.
`lead`	Audio con la pista principal aislada: la voz.	De aquí salen los tiempos y los tonos. Si está vacío se genera a partir de `audio`.
`accompaniment`	Audio del acompañamiento.	Es lo que sonará en el karaoke y de él se calcula el BPM. Si está vacío se genera a partir de `audio`.
`lyrics`	Fichero de texto UTF-8 con la letra.	Si está vacío, se usa como letra la transcripción de Whisper. Cada línea del fichero es una línea del karaoke; las vacías se ignoran. Conviene no pasar de 45 caracteres por línea.
`language`	Código de idioma de dos letras.	`es`, `en`, `fr`, `de`, `it`, `pt`... Decide el silabeo y la etiqueta `#LANGUAGE`. Si está vacío se deduce de la letra con el LLM (sin LLM, de la voz con Whisper).
`range`	Rango en el que se quiere cantar.	Dos notas separadas por `_`, por ejemplo `do3_mi4`. Si está vacío no se transporta.

Las rutas pueden ser absolutas o relativas a la carpeta del cfg. El script se detiene con un mensaje claro si falta un campo, si un fichero no existe o si `lyrics` apunta a un audio.

El campo range

Cada nota se escribe con su nombre en español (`do`, `re`, `mi`, `fa`, `sol`, `la`, `si`), una alteración opcional (`#` o `b`) y la octava, con `do4` como do central del piano. `do3_mi4` va del do por debajo del do central al mi por encima; `fa#2_sib3` también es válido.

Con el rango, el script busca la transposición que deja más tiempo cantado dentro de él y genera una segunda versión de la canción. El audio se mueve como mucho seis semitonos: si hace falta más, se canta una octava por encima o por debajo, que no necesita otro acompañamiento y que UltraStar puntúa igual. Si la canción ya encaja, o transportarla apenas mejora, no se genera nada más.

Campos opcionales

Campo	Efecto
`artist`, `title`	Fijan `#ARTIST` y `#TITLE`. Si faltan, los propone el LLM a partir de `name` y del comienzo de la letra.
`genre`, `year`	Fijan `#GENRE` y `#YEAR`. Si faltan, los propone el LLM; si no los conoce, la etiqueta se omite.
`creator`	Fija `#CREATOR`. Tiene prioridad sobre la opción `--creator`.
`cover`	Imagen de carátula. Se copia a `songs/` y da la etiqueta `#COVER`. La rellena el script si el audio lleva una carátula incrustada.
`video`	Vídeo de fondo. Se copia a `songs/` y da la etiqueta `#VIDEO`. Lo rellena el script si el fichero de audio contiene una pista de vídeo.
`intrument`, `score`	Reservados para más adelante. Hoy se leen y se ignoran.

Los metadatos que propone el LLM no están verificados y el informe lo avisa. Para una canción definitiva es preferible escribirlos en el cfg.

2. Fichero txt de UltraStar

El resultado es `songs/<name>.txt`, en UTF-8 sin BOM. Tiene una cabecera de etiquetas `#ETIQUETA:valor`, una línea por nota y una E final. Este es el comienzo del fichero generado para la canción de ejemplo (artista y título son la propuesta, errónea aquí, del LLM):

```
#ARTIST:Fenesta
#TITLE:Che lucive
#LANGUAGE:Italian
#GENRE:Folk
#CREATOR:karaokame
#MP3:Fenesta_che_lucive.mp3
#COVER:Fenesta_che_lucive.jpg
#BPM:323,00
#GAP:27160,00
: 0 4 -9 Fe
: 4 7 -7 ~
: 11 9 -7 ne
: 20 10 -5 sta
...
E
```

Cabecera

Etiqueta	Significado	Cómo la rellena karaokame
`#ARTIST`	Intérprete de la canción.	Campo `artist` del cfg o propuesta del LLM.
`#TITLE`	Título de la canción.	Campo `title` del cfg o propuesta del LLM.
`#LANGUAGE`	Idioma principal, en inglés salvo «Español».	A partir de `language`: `es`, `ca`, `gl` y `eu` dan «Español»; el resto, English, French, German, Italian, Portuguese.
`#GENRE`	Género musical.	Campo `genre` o propuesta del LLM. Se omite si no se conoce.
`#YEAR`	Año de publicación.	Campo `year` o propuesta del LLM. Se omite si no se conoce.
`#CREATOR`	Autor del txt.	Campo `creator` u opción `--creator`.
`#MP3`	Fichero de audio que suena.	`<name>.mp3`, la copia del acompañamiento.
`#COVER`	Carátula.	`<name>.jpg`, copia del campo `cover`. Solo si el cfg lo tiene.
`#VIDEO`	Vídeo de fondo.	`<name>.mp4`, copia del campo `video`. Solo si el cfg lo tiene.
`#BPM`	Velocidad de la rejilla de beats.	Tempo detectado en el acompañamiento, duplicado hasta quedar entre 200 y 400. Con coma decimal.
`#GAP`	Milisegundos hasta el beat 0.	Instante en que empieza la primera nota. Con coma decimal.

No se generan `#BACKGROUND`, `#MEDLEYSTARTBEAT` ni `#MEDLEYENDBEAT`; hay que añadirlas a mano. El nombre del mp3 sigue el `name` del cfg, no la convención «Artista - Título.mp3».

Líneas de nota

Cada nota es una línea con cinco campos separados por espacios, por ejemplo `: 20 10 -5 sta`:

Campo	Ejemplo	Significado
Tipo	`:`	`:` normal, `*` dorada, `F` libre sin puntuar, `R` rap, `G` rap dorada. karaokame solo escribe notas normales.
Beat de inicio	`20`	Posición de la nota, en beats desde el GAP.
Duración	`10`	Longitud en beats. Como mínimo 1.
Tono	`-5`	Semitonos respecto al do central (C4 = 0). El juego compara sin tener en cuenta la octava.
Texto	`sta`	Sílaba que se canta.

Un beat del txt dura `15 / BPM` segundos, es decir, un cuarto del pulso indicado en `#BPM`. El instante real de una nota es `GAP + beat × 15000 / BPM` milisegundos. Con BPM 323 cada beat dura 46,4 ms.

La versión transportada, `songs/<name>_X.txt` (por ejemplo `_-4`), es el mismo fichero con el tono de todas las notas desplazado, `#MP3` apuntando a `<name>_X.mp3` y `(X)` añadido al título para distinguirla en el juego. Tiempos, BPM y GAP no cambian.

Otras líneas y convenciones

Elemento	Significado
`- N`	Cambio de línea en el beat `N`. Se coloca entre el final de una línea y el comienzo de la siguiente.
`E`	Fin de la canción.
Espacio final	La última sílaba de cada palabra lleva un espacio detrás, salvo al final de la línea. Así el juego sabe dónde separar las palabras.
Sinalefa	Dos vocales de palabras contiguas cantadas juntas se escriben en una sola nota: `go el`, `te ha`.
`~`	Cambio de tono dentro de una sílaba: la primera nota lleva el texto y las siguientes solo la virgulilla.

3. Configuración del script

El script se lanza desde la carpeta del proyecto con el intérprete del entorno virtual:

```
.venv/bin/python bin/karaokame.py [opciones] [config/cancion.cfg ...]
```

El argumento posicional `configs` es la lista de ficheros cfg a procesar. Si alguna canción falla, el script sigue con las demás y termina con código de salida 1.

Sin argumentos: la carpeta data/

Si no se indica ningún cfg, el script busca la canción en `data/`, donde debe haber una sola, crea su cfg en `config/` y la procesa.

Qué busca	Cómo lo reconoce
La canción	Cualquier fichero con una pista de audio, comprobado con `ffprobe` y no por la extensión: mp3, wav, flac, m4a, mp4, mkv, webm... Si trae vídeo, se extrae y se usa como `#VIDEO`; si trae una carátula incrustada, como `#COVER`.

Qué busca	Cómo lo reconoce
La letra	El `.txt` con el mismo nombre que la canción o, si no existe, el único `.txt` de la carpeta. Si no hay ninguno, se transcribe de la voz y queda en `work/<name>/lyrics_whisper.txt` para corregirla.
Lo que ignora	Los ficheros generados por los propios scripts (`*_V`, `*_A`, `*_video`), las imágenes y cualquier fichero sin audio.
La plantilla	Si existe `config/template.cfg`, el cfg nuevo parte de él: sirve para fijar valores comunes como `range` o `creator`. Un cfg que ya existe no se sobrescribe.
Varias canciones	Se procesa la única que aún no tenga su txt en `songs/`. Si hay varias pendientes, o varias ya generadas, el script se detiene y pide dejar solo una o indicar el cfg.

Para cada canción hace, por orden: separar voz y acompañamiento si faltan, transcribir la letra si falta, detectar el idioma si falta, generar el txt y el mp3, y transportar al rango si el cfg tiene `range`.

Opción	Por defecto	Descripción
`--whisper-model`	`medium`	Modelo de faster-whisper: `small`, `medium`, `large-v3`... Los mayores oyen mejor la voz cantada, pero tardan más y ocupan más disco. Se descargan la primera vez.
`--crepe-model`	`full`	`full` o `tiny`. `tiny` es mucho más rápido y algo menos preciso en el tono.
`--device`	`auto`	`auto`, `cpu` o `cuda`. `auto` usa la GPU si PyTorch la detecta y, si no, la CPU.
`--no-llm`	desactivada	No usa Ollama. No se proponen metadatos ni se revisan las líneas dudosas; el resto funciona igual.
`--llm-model`	`qwen3:4b-instruct-2507-q4_K_M`	Modelo de Ollama para metadatos y revisión.
`--ollama-url`	`http://localhost:11434`	Dirección del servidor de Ollama.
`--no-sinalefa`	desactivada	Escribe una nota por sílaba aunque dos vocales de palabras contiguas se canten juntas.
`--no-melisma`	desactivada	No parte las sílabas con cambios de tono en notas ~; cada sílaba queda como una sola nota.
`--creator`	`karaokame`	Valor de `#CREATOR` cuando el cfg no trae el campo `creator`.
`--uvr-dir`	`ultimatevocalremovergui/`	Carpeta con la copia de Ultimate Vocal Remover que se usa para separar la voz (modelo MDX23C-InstVoc HQ, segmento 256, solape 8).
`--force`	desactivada	Ignora la caché: vuelve a ejecutar Whisper y CREPE y repite la separación de voz.

Ejemplos

```
# La canción que haya en data/, con los valores por defecto
.venv/bin/python bin/karaokame.py

# Una canción, con el modelo grande de Whisper y sin LLM
.venv/bin/python bin/karaokame.py --whisper-model large-v3 --no-llm config/cancion.cfg

# Prueba rápida de la canción de data/: CREPE pequeño, una nota por sílaba
.venv/bin/python bin/karaokame.py --crepe-model tiny --no-melisma --no-sinalefa
```

Salidas y caché

Fichero	Contenido
`config/<name>.cfg`	Cfg creado para la canción de data/.
`songs/<name>.txt`	Canción de UltraStar.
`songs/<name>.mp3`	Copia del acompañamiento.
`songs/<name>.jpg / .mp4`	Carátula y vídeo, si el cfg los indica.
`songs/<name>_X.txt / .mp3`	Versión transportada X semitonos, si hay rango y hace falta transportar.
`<audio>_V.mp3 / _A.mp3`	Voz y acompañamiento separados, junto al fichero de audio.
`work/<name>/report.md`	Informe: calidad de la alineación, avisos y líneas que conviene revisar a mano.
`work/<name>/whisper_<modelo>.json`	Caché de las palabras oídas por Whisper.
`work/<name>/pitch_<modelo>.npz`	Caché del análisis de tono de CREPE.

La caché depende de la pista de voz, del idioma y del modelo. Mientras no cambien, las ejecuciones siguientes tardan segundos, lo que permite probar opciones como las de sinalefa o melisma sin repetir el análisis. Con `--force` se recalcula todo. Como referencia, en CPU una canción de 3:40 tarda unos 7 minutos en CREPE full y entre 2 y 3 en Whisper medium.